

ĐẠI HỌC QUỐC GIA, THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Khai Phá Dữ Liệu (CO3029)

Báo cáo Bài tập lớn

"Hand Gesture Recognition System"

GVHD: Đỗ Thanh Thái

SV thực hiện: Đào Quang Dương L02 - 2310579
Võ Thanh Đạt L01 - 2310717
Hà Bảo Nhi L02 - 2312496

Nhóm: 07

Source code: [Hand Gesture Recognition](#).

TP Hồ Chí Minh, Tháng 4 Năm 2026



Nội dung

Danh sách hình ảnh minh họa	3
Danh sách bảng	3
Thành viên và khối lượng công việc	3
1 Giới thiệu	4
1.1 Tổng quan về bài toán	4
1.2 Mục tiêu và Phạm vi dự án	4
1.3 Ý nghĩa thực tiễn của bài toán	5
2 Dữ liệu và Tiền xử lý dữ liệu	6
2.1 Tổng quan bộ dữ liệu	6
2.2 Phân tích Khám phá Dữ liệu (EDA)	6
2.2.1 Phân bố nhân dữ liệu (Target Distribution)	6
2.2.2 Phân tích đặc trưng không gian và điểm ngoại lai (Outlier Analysis)	7
2.2.3 Phân tích tương quan đặc trưng (Feature Correlation)	8
2.2.4 Trực quan hóa không gian phân cụm (t-SNE)	9
2.3 Quy trình Tiền xử lý dữ liệu (Data Preprocessing)	10
2.3.1 Pipeline Tiền xử lý Cử chỉ tĩnh (Static Gesture Preprocessing)	10
2.3.2 Pipeline Tiền xử lý Cử chỉ động (Dynamic Gesture Preprocessing)	10
2.3.3 Đánh giá định lượng hiệu quả Tiền xử lý	11
3 Cơ sở lý thuyết và Kiến trúc Thuật toán	12
3.1 Thuật toán XGBoost (Extreme Gradient Boosting)	12
3.1.1 Nguyên lý hoạt động	12
3.1.2 Hàm mục tiêu và Cơ chế tối ưu toán học	12
3.2 Mạng nơ-ron hồi quy GRU (Gated Recurrent Unit)	12
3.2.1 Vấn đề của mạng RNN và giải pháp GRU	12
3.2.2 Cơ sở toán học của GRU	13
3.3 Trích xuất không gian với MediaPipe Hand Landmarker	13
3.4 Thuật toán định vị đối tượng YOLOv8	14
3.4.1 Cơ sở lý thuyết YOLOv8	14
3.4.2 Đặc điểm Anchor-Free	14
4 Thiết lập thực nghiệm	15
4.1 Cấu trúc tập dữ liệu và Kỹ thuật Tăng cường (Augmentation)	15
4.1.1 Chiến lược phân chia Hold-out (Data Splitting)	15
4.1.2 Kỹ thuật Sinh dữ liệu nhân tạo (Data Augmentation)	15
4.2 Không gian tìm kiếm siêu tham số (Hyperparameter Space)	15
4.3 Quá trình tinh chỉnh và Kết quả trên tập Validation	16
4.3.1 Tinh chỉnh tự động cho mô hình tĩnh (XGBoost)	16
4.3.2 Tinh chỉnh thực nghiệm cho mạng động (GRU)	16
4.3.3 Phân tích và Lựa chọn	17



5	Kết quả và phân tích đánh giá	19
5.1	Tổng quan kết quả trên tập kiểm tra (Test Set)	19
5.2	Phân tích chi tiết mô hình phân loại Cờ chỉ tĩnh (XGBoost)	19
5.3	Phân tích chi tiết mạng học sâu phân loại Hành động (GRU)	20
5.4	So sánh đối chiếu và Đánh giá tổng kết	22
6	Kết luận	23
6.1	Tổng kết quy trình nghiên cứu và Đóng góp	23
6.2	Đánh giá hiệu năng hệ thống	23
6.3	Hạn chế của nghiên cứu	24
6.4	Hướng phát triển tương lai	24
6.5	Ý nghĩa thực tiễn	24
7	Tài liệu tham khảo	25

Danh sách hình ảnh minh họa

1	Biểu đồ phân bố số lượng mẫu trên từng lớp cờ chỉ tĩnh	6
2	Biểu đồ Boxplot phân tích độ phân tán của tọa độ y tại ngón trỏ (p8_y)	7
3	Ma trận tương quan Pearson giữa tọa độ các điểm mốc (landmarks)	8
4	Phép chiếu t-SNE giảm chiều dữ liệu từ 42 chiều xuống không gian 2D	9
5	Dashboard tổng quan quy trình Tiền xử lý	11
6	Dashboard đánh giá tổng quan mô hình XGBoost (Power BI)	19
7	Đánh giá Kết hợp (Composite Evaluation) của mô hình XGBoost trên tập Test	20
8	Dashboard đánh giá mô hình GRU và Hiệu năng Vận hành hệ thống	21
9	Đánh giá Kết hợp (Composite Evaluation) của mạng GRU trên tập Test	21

Danh sách bảng

1	Thành viên và khối lượng nhiệm vụ	3
2	So sánh hiệu năng hệ thống chịu tác động của Pipeline Tiền xử lý	11
3	Tập hợp các siêu tham số được đưa vào không gian tìm kiếm	16
4	Kết quả đánh giá độ trễ các cấu hình mạng GRU	17
5	Bộ siêu tham số tối ưu được lựa chọn cho giai đoạn kiểm thử	18
6	Tổng hợp hiệu năng các thuật toán trên tập kiểm tra (Unseen Data)	19
7	Tổng quan hiệu năng tổng hợp của toàn bộ hệ thống	23



Thành viên và khối lượng nhiệm vụ

STT	Họ & tên	MSSV	Nhiệm vụ đảm nhiệm	% Hoàn thành
1	Đào Quang Dương	2310579	- Lập trình và huấn luyện mô hình nhận diện (XGBoost, mạng GRU). - Tích hợp luồng camera xử lý thời gian thực (Real-time). - Tinh chỉnh siêu tham số (Hyperparameter Tuning).	100%
2	Võ Thanh Đạt	2310717	- Xử lý số liệu, xuất các ma trận nhầm lẫn và metrics đánh giá. - Thiết kế và xây dựng Dashboard trực quan hóa trên Power BI. - Hỗ trợ phân tích lỗi (Error Analysis) từ các biểu đồ thực nghiệm.	100%
3	Hà Bảo Nhi	2312496	- Khai phá dữ liệu (EDA) và Tiền xử lý (Preprocessing) chuyên sâu. - Tham gia lập trình thuật toán và trích xuất đặc trưng tay. - Soạn thảo báo cáo, tổng hợp và phân tích kết quả thực nghiệm.	100%

Table 1: Thành viên và khối lượng nhiệm vụ

1 Giới thiệu

1.1 Tổng quan về bài toán

Trong thời đại công nghệ số hiện nay, việc tương tác giữa con người và các thiết bị thông minh ngày càng trở nên đa dạng và phong phú. Từ các phương thức truyền thống như bàn phím, chuột, cho đến các công nghệ tiên tiến như nhận dạng giọng nói, cử chỉ tay đã dần trở thành xu hướng phổ biến trong lĩnh vực điều khiển thiết bị thông minh. Đặc biệt trong bối cảnh Internet of Things (IoT) và Smart Home đang phát triển mạnh mẽ, nhu cầu về các giao diện tự nhiên, trực quan và không cần chạm (touchless) ngày càng được chú trọng.

Bài toán **Nhận diện cử chỉ tay (Hand Gesture Recognition)** là một trong những bài toán cốt lõi trong lĩnh vực Thị giác máy tính (Computer Vision) và Học sâu (Deep Learning). Mục tiêu của bài toán là xây dựng một hệ thống có khả năng:

- Nhận diện chính xác các cử chỉ tay của người dùng từ dữ liệu hình ảnh hoặc video.
- Phân loại các cử chỉ thành các nhãn tương ứng (ví dụ: một, hai, ba ngón tay, hình V victory, ...).
- Xử lý trong thời gian thực (real-time) để đáp ứng nhu cầu tương tác ngay lập tức.

Hệ thống nhận diện cử chỉ tay được ứng dụng rộng rãi trong nhiều lĩnh vực như: điều khiển robot, điều khiển thiết bị gia đình thông minh, điều hướng trong giao diện người-máy (HMI), hỗ trợ người khuyết tật giao tiếp, và các trò chơi tương tác. Với sự phát triển của các mô hình học sâu và sự tiến bộ trong phần cứng máy tính, bài toán này đã đạt được nhiều tiến bộ đáng kể trong những năm gần đây.

Mục tiêu nghiên cứu

Nghiên cứu này tập trung vào việc xây dựng một **Hệ thống nhận diện cử chỉ tay thời gian thực** kết hợp giữa nhận diện cử chỉ tĩnh (static gestures) và cử chỉ động (dynamic motions). Hệ thống sử dụng kết hợp các mô hình học máy truyền thống (XGBoost) cho bài toán phân loại nhanh và mạng nơ-ron hồi quy (GRU) cho bài toán nhận diện chuyển động. Điểm đặc biệt của hệ thống là khả năng theo dõi đồng thời nhiều người (multi-person tracking) và xử lý nhiễu hiệu quả để đảm bảo độ tin cậy trong điều kiện thực tế.

1.2 Mục tiêu và Phạm vi dự án

Mục tiêu chính của dự án là xây dựng một hệ thống nhận diện cử chỉ tay hoàn chỉnh với các yêu cầu cụ thể sau:

Về chức năng: Hệ thống cần hỗ trợ nhận diện hai loại cử chỉ:

- **Cử chỉ tĩnh (Static Gestures):** Bao gồm 5 loại cử chỉ cơ bản là "một ngón" (one), "hai ngón" (two), "ba ngón" (three), "ngón victory" (seven), và "mở/đóng bàn tay" (open/close). Các cử chỉ này được phân loại dựa trên vị trí của 21 điểm đặc trưng (landmarks) của bàn tay.
- **Cử chỉ động (Dynamic Motions):** Bao gồm hai loại hành động là "vỗ tay" (clap) và "lắc tay" (shake). Các hành động này được nhận diện dựa trên chuỗi thời gian của các điểm landmark qua 50 khung hình liên tiếp.

Về hiệu năng: Hệ thống phải đạt được tốc độ xử lý thời gian thực với tốc độ khung hình (FPS) tối thiểu là 30 FPS, đảm bảo trải nghiệm người dùng mượt mà. Đồng thời, hệ thống cần có khả năng xử lý đồng thời nhiều người trong cùng một khung hình.

Về độ tin cậy: Hệ thống cần có cơ chế lọc nhiễu hiệu quả thông qua các kỹ thuật như Buffer Voting (20/30 khung hình đồng thuận), Hold-to-confirm (giữ tư thế 1 giây), và ngưỡng tin cậy (confidence threshold) 85%. Ngoài ra, hệ thống cần có cơ chế Smart Cooldown để tránh các lệnh trùng lặp với thời gian nghỉ 1.5 giây giữa hai lần kích hoạt.

Về dữ liệu: Dự án sử dụng dữ liệu được thu thập và gán nhãn thủ công, bao gồm hình ảnh tĩnh cho cử chỉ tĩnh và video cho cử chỉ động. Quá trình tiền xử lý dữ liệu (Data Preprocessing) đóng vai trò quan trọng trong việc đảm bảo chất lượng dữ liệu đầu vào cho mô hình.

Phạm vi của dự án tập trung vào việc xây dựng và đánh giá hệ thống nhận diện cử chỉ trong điều kiện phòng thí nghiệm với camera web thông thường. Hệ thống được thiết kế với mục tiêu cuối cùng là ứng dụng trong điều khiển thiết bị thông minh trong gia đình (Smart Home Control).

1.3 Ý nghĩa thực tiễn của bài toán

Bài toán nhận diện cử chỉ tay mang lại nhiều ý nghĩa quan trọng trong thực tiễn:

- **Điều khiển thiết bị thông minh:** Trong môi trường Smart Home, người dùng có thể điều khiển các thiết bị như đèn, quạt, điều hòa, TV mà không cần sử dụng remote hoặc chạm vào các bề mặt. Điều này đặc biệt hữu ích trong bối cảnh dịch bệnh hoặc khi tay người dùng đang bận.
- **Giao diện người-máy tự nhiên:** Cử chỉ tay là một trong những hình thức giao tiếp tự nhiên nhất của con người. Việc tích hợp nhận diện cử chỉ vào các hệ thống giúp giao diện trở nên thân thiện và trực quan hơn, đặc biệt cho người dùng cao tuổi hoặc trẻ em.
- **Hỗ trợ người khuyết tật:** Hệ thống có thể được ứng dụng để hỗ trợ người khiếm thị hoặc người khuyết tật vận động trong việc giao tiếp và điều khiển thiết bị, mở rộng khả năng tiếp cận công nghệ cho những đối tượng này.
- **Nền tảng cho các ứng dụng khác:** Kỹ thuật và phương pháp được phát triển trong bài toán này có thể được mở rộng và áp dụng cho các bài toán nhận diện hành động (Action Recognition), nhận diện khuôn mặt (Face Recognition), hoặc các hệ thống tương tác thực tế ảo (VR/AR).

Với những ý nghĩa thực tiễn như vậy, việc nghiên cứu và xây dựng hệ thống nhận diện cử chỉ tay không chỉ có giá trị về mặt học thuật mà còn có tiềm năng ứng dụng rộng rãi trong đời sống hàng ngày. Trong báo cáo này, nghiên cứu sẽ trình bày chi tiết về quá trình xây dựng hệ thống, từ giai đoạn thu thập và tiền xử lý dữ liệu, đến việc lựa chọn và huấn luyện mô hình, cũng như đánh giá hiệu năng của hệ thống.

2 Dữ liệu và Tiền xử lý dữ liệu

Phần này trình bày chi tiết về bộ dữ liệu, các phương pháp Phân tích Khám phá Dữ liệu (EDA) và quy trình tiền xử lý. Trong bài toán nhận diện qua camera, dữ liệu hình ảnh ban đầu là dữ liệu thô (raw data), phi cấu trúc và chứa nhiều độ nhiễu. Do đó, việc chuyển đổi, khám phá và tiền xử lý đóng vai trò "sống còn" quyết định đến khả năng tổng quát hóa của mô hình.

2.1 Tổng quan bộ dữ liệu

Dữ liệu được thu thập thủ công thông qua module `collect_data.py`. Thay vì đưa nguyên bức ảnh vào mô hình, nhóm sử dụng MediaPipe như một công cụ trích chọn đặc trưng (Feature Extraction) để chuyển đổi dữ liệu phi cấu trúc thành dữ liệu bảng (Tabular data), tập trung hoàn toàn vào hình thái học khung xương.

Tổng cộng hệ thống đã trích xuất thành công **14,467 mẫu dữ liệu hợp lệ**, chia làm hai nhánh:

- **Cử chỉ tĩnh (Static):** 6 lớp (one, two, three, open_close, rotate, victory). Mỗi mẫu là một vector 42 chiều (tọa độ x, y của 21 điểm landmark).
- **Cử chỉ động (Dynamic):** 3 lớp (clap, shake, none). Mỗi mẫu là một chuỗi (sequence) gồm 50 khung hình liên tiếp, mỗi khung hình chứa 252 đặc trưng (tọa độ và vận tốc).

2.2 Phân tích Khám phá Dữ liệu (EDA)

Quá trình EDA được thực hiện nhằm đánh giá phân phối, phát hiện nhiễu không gian và tìm ra các đặc trưng mang tính "chìa khóa" (key features) để phân biệt các lớp cử chỉ.

2.2.1 Phân bố nhân dữ liệu (Target Distribution)

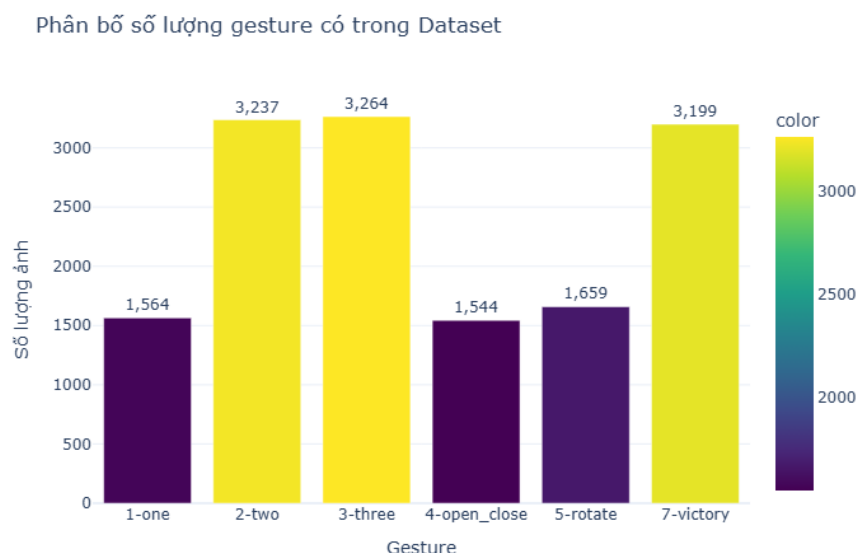


Figure 1: Biểu đồ phân bố số lượng mẫu trên từng lớp cử chỉ tĩnh

Phân tích và Chẩn đoán:

- Tổng quan 14,467 mẫu được phân bố tương đối ổn định nhưng vẫn tồn tại sự chênh lệch nhẹ giữa các nhóm cử chỉ.
- Nhóm cử chỉ tĩnh (như `two`, `three`, `victory`) có số lượng dồi dào, đạt trên 3,000 mẫu cho mỗi lớp.
- Nhóm cử chỉ chuyển động ngón tay (`open_close`, `rotate`) dao động ở mức thấp hơn, khoảng 1,500 - 1,700 mẫu.

Ý nghĩa thực tiễn:

- Tỷ lệ chênh lệch lớn nhất giữa lớp đa số và thiểu số rơi vào khoảng 2:1. Trong học máy, đây là mức độ mất cân bằng nhẹ, nằm trong ngưỡng hoàn toàn an toàn.
- Kích thước tập dữ liệu của lớp thiểu số vẫn đủ lớn ($> 1,500$ mẫu) để thuật toán phân loại XGBoost học được ranh giới quyết định mà không bị thiên vị (bias).
- **Định hướng:** Nhóm quyết định giữ nguyên phân phối tự nhiên này, không can thiệp bằng các kỹ thuật sinh mẫu (như SMOTE) để tránh tạo ra nhiễu nhân tạo.

2.2.2 Phân tích đặc trưng không gian và điểm ngoại lai (Outlier Analysis)

Biểu đồ 2: So sánh phân bố tọa độ P8_Y giữa các loại cử chỉ

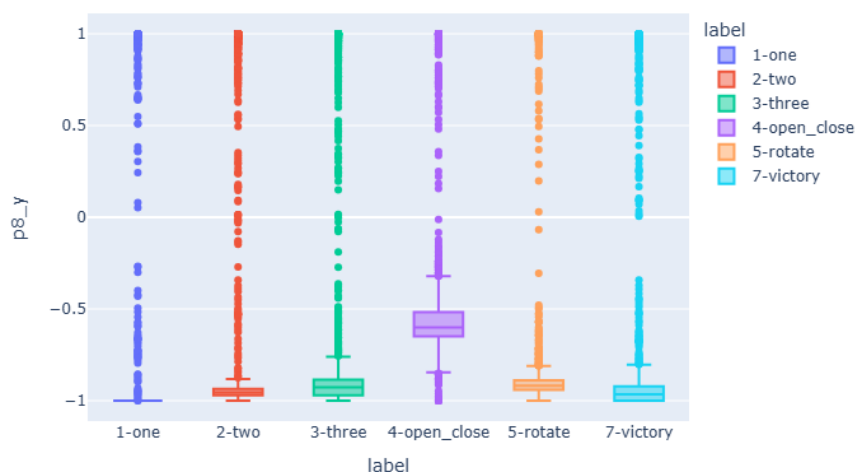


Figure 2: Biểu đồ Boxplot phân tích độ phân tán của tọa độ y tại ngón trỏ (`p8_y`)

Phân tích và Chẩn đoán:

- Biểu đồ Boxplot thể hiện dải giá trị phân tán cực kỳ rộng (từ 0.1 đến 0.9) đối với các điểm landmark trước khi được chuẩn hóa.

- Xuất hiện hàng loạt các điểm ngoại lai (outliers) nằm ngoài vùng tứ phân vị (IQR).
- Hiện tượng này không xuất phát từ việc gán nhãn sai, mà đại diện cho **độ nhiễu không gian tự nhiên** khi thu thập dữ liệu bằng camera (người dùng đưa tay quá gần/quá xa hoặc đứng lệch sang các mép màn hình).

Ý nghĩa thực tiễn:

- Biểu đồ này là bằng chứng rõ ràng nhất cho thấy dữ liệu thô (raw data) không thể sử dụng trực tiếp để huấn luyện.
- **Định hướng:** Bắt buộc phải tích hợp bước tịnh tiến tọa độ (Translation) và tỷ lệ hóa (Scaling) trong quy trình tiền xử lý nhằm triệt tiêu phương sai do vị trí đứng của người dùng gây ra.

2.2.3 Phân tích tương quan đặc trưng (Feature Correlation)

Landmark Feature Correlation Heatmap

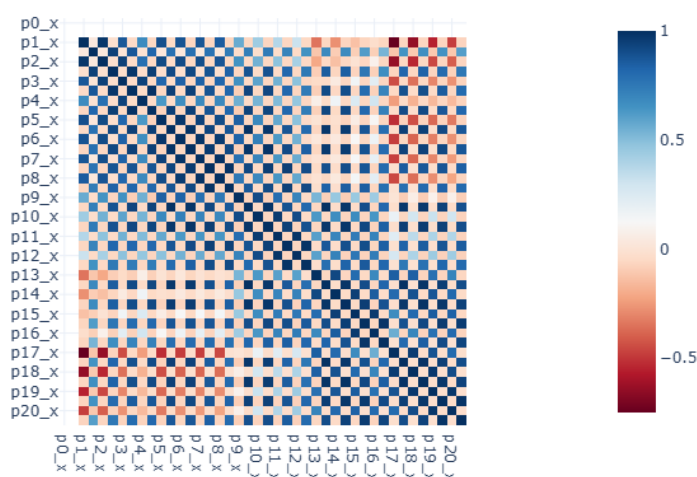


Figure 3: Ma trận tương quan Pearson giữa tọa độ các điểm mốc (landmarks)

Phân tích và Chẩn đoán:

- Ma trận hiển thị các vùng màu sáng đậm, minh chứng cho hiện tượng đa cộng tuyến (multicollinearity) rất cao trong dữ liệu.
- Các đốt ngón tay kề nhau (ví dụ: p4-p5 trên ngón cái, p8-p9 trên ngón trỏ) có hệ số tương quan Pearson $r > 0.85$, do sự chuyển động của một khớp xương thường kéo theo sự dịch chuyển của khớp liền kề.

Ý nghĩa thực tiễn:

- Dù mang tính trùng lặp (redundant), nhóm nhận định cấu trúc vì mô này chính là "chìa khóa" để phân biệt hai lớp có hình thái gần giống nhau nhất là 2-two và 7-victory (phân biệt qua góc độ mở của các đốt ngón tay).
- **Định hướng:** Quyết định không sử dụng PCA để nén dữ liệu mà giữ nguyên toàn bộ không gian 42 chiều, tận dụng cơ chế phân nhánh tự động (tree-building) dựa trên Information Gain của thuật toán XGBoost.

2.2.4 Trực quan hóa không gian phân cụm (t-SNE)

Biểu đồ t-SNE: Phân cụm toàn bộ các loại cử chỉ tay

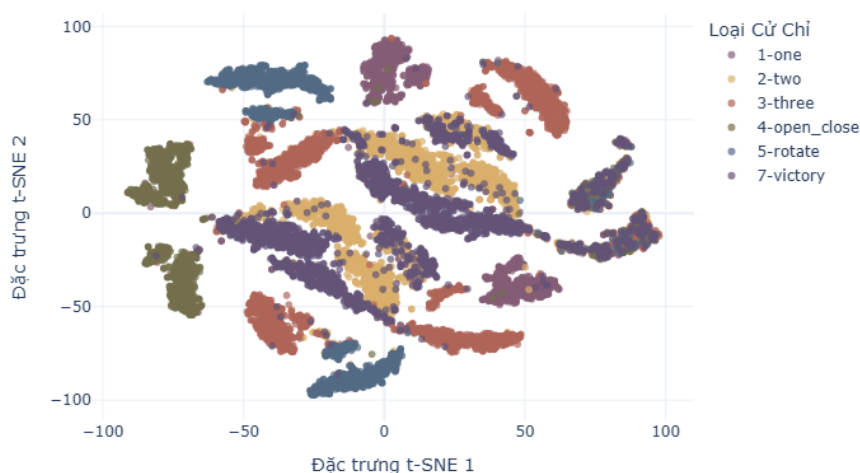


Figure 4: Phép chiếu t-SNE giảm chiều dữ liệu từ 42 chiều xuống không gian 2D

Phân tích và Chẩn đoán:

- Biểu đồ t-SNE cho thấy các dải màu (tương ứng với các lớp cử chỉ) tụ lại thành các cụm (clusters) tương đối rắn chắc.
- Ranh giới giữa các lớp hiển thị khá rõ ràng, mức độ phân tách không gian tốt dù vẫn còn một số vùng chồng lấp nhỏ ở vùng biên.

Ý nghĩa thực tiễn:

- Xác nhận bộ đặc trưng 42 chiều trích xuất từ thuật toán MediaPipe có đủ năng lực phân biệt (discriminative power) đối với bài toán này.
- **Định hướng:** Khẳng định bài toán có thể được giải quyết với độ chính xác cao bằng các mô hình Machine Learning phân loại truyền thống (như XGBoost), giúp tiết kiệm đáng kể tài nguyên phần cứng và đảm bảo yêu cầu Real-time (> 30 FPS) thay vì phải dùng mạng CNN công kênh.

2.3 Quy trình Tiền xử lý dữ liệu (Data Preprocessing)

Dựa trên các chẩn đoán từ bước EDA, hệ thống tiến hành bước tiền xử lý nhằm chuẩn hóa dữ liệu. Nếu sử dụng trực tiếp tọa độ thô, dữ liệu sẽ bị "tàng hình" hoặc sai lệch hoàn toàn khi khoảng cách và góc đứng của người dùng thay đổi.

2.3.1 Pipeline Tiền xử lý Cử chỉ tĩnh (Static Gesture Preprocessing)

Nhóm áp dụng luồng xử lý không gian 2 bước cho 21 điểm landmark ($21 \times 2 = 42$ đặc trưng số):

Quy trình Chuẩn hóa Không gian Bàn tay

Bước 1: Tịnh tiến gốc tọa độ (Translation - Dời tâm)

Thay vì lấy gốc tọa độ (0,0) tại góc màn hình camera, hệ thống dời tâm về điểm cổ tay (Landmark 0) của người dùng:

$$x'_i = x_i - x_0, \quad y'_i = y_i - y_0 \quad \forall i \in [1, 20] \quad (1)$$

Tác dụng: Loại bỏ nhiễu vị trí. Bàn tay ở góc trái hay góc phải khung hình đều cho ra cùng một ma trận tọa độ tương đối.

Bước 2: Tỷ lệ hóa Min-Max (Normalization)

Tính toán khoảng cách lớn nhất từ cổ tay đến các điểm mốc, sau đó ép toàn bộ tọa độ về dải giá trị chuẩn $[-1, 1]$:

$$x''_i = \frac{x'_i}{\max(|x'|, |y'|)}, \quad y''_i = \frac{y'_i}{\max(|x'|, |y'|)} \quad (2)$$

Tác dụng: Loại bỏ nhiễu khoảng cách. Dù người dùng lùi xa camera (bàn tay nhỏ lại) hay tiến lại gần (bàn tay to ra), vector đặc trưng đầu vào vẫn hoàn toàn bất biến.

2.3.2 Pipeline Tiền xử lý Cử chỉ động (Dynamic Gesture Preprocessing)

Đối với dữ liệu chuỗi thời gian của mạng GRU, sự liên mạch đóng vai trò cốt lõi. Nhóm áp dụng luồng xử lý 4 bước:

- Nội suy khuyết thiếu (Missing Value Imputation):** Quá trình vung tay nhanh (motion blur) khiến MediaPipe mất dấu (drop frames). Hệ thống áp dụng nội suy tuyến tính để điền bù tọa độ, đảm bảo chuỗi không bị gãy.
- Căn chỉnh độ dài chuỗi (Sequence Resizing):** Các hành động võ tay có thể kéo dài từ 30 đến 70 frames. Nhóm sử dụng kỹ thuật nội suy thời gian (Temporal Interpolation) để co/giãn mọi hành động về kích thước chuẩn hóa cố định là $N = 50$ frames.
- Chuẩn hóa Không gian:** Áp dụng lại phương pháp dời tâm và tỷ lệ hóa của cử chỉ tĩnh lên từng frame trong chuỗi.
- Trích xuất Vận tốc (Velocity Encoding):** Vị trí tĩnh là chưa đủ để phân biệt hành động vẩy tay. Nhóm tính toán thêm vector vận tốc bằng đạo hàm rời rạc: $v_t = x_t - x_{t-1}$. Vector đặc trưng cuối cùng được mở rộng lên 252 chiều/frame (kết hợp cả tọa độ chuẩn hóa và vận tốc).

2.3.3 Đánh giá định lượng hiệu quả Tiền xử lý

Để chứng minh luận điểm "Tiền xử lý là yếu tố sống còn", nhóm đã tiến hành đối chiếu trực quan trên Dashboard và huấn luyện hai mô hình XGBoost song song trên tập dữ liệu thô và tập dữ liệu đã qua xử lý.

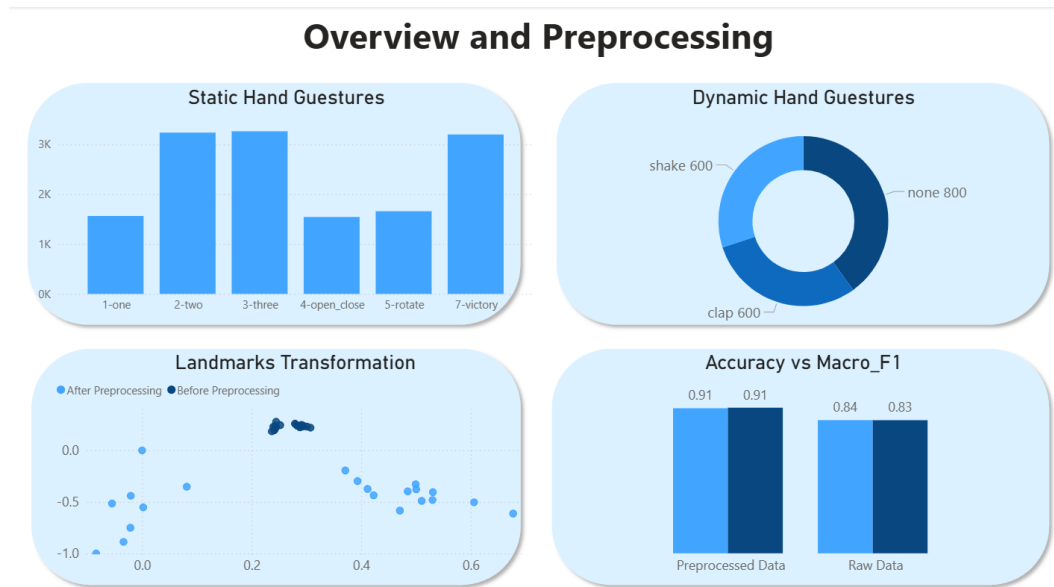


Figure 5: Dashboard tổng quan quy trình Tiền xử lý

Phân tích từ Dashboard (Hình 5):

- Biểu đồ Scatter (Landmarks Transformation):** Minh họa trực quan sự lộn xộn của không gian dữ liệu. Trước khi xử lý, các điểm landmark phân tán hỗn loạn do nhiễu không gian. Sau khi áp dụng dời tâm và tỷ lệ hóa, dữ liệu được gom lại thành các cấu trúc khung xương (skeleton) nhất quán, triệt tiêu hoàn toàn sự phụ thuộc vào khoảng cách người dùng.
- Biểu đồ Bar Chart (Performance Impact):** Ghi nhận sự tăng trưởng rõ rệt.

Cấu hình mô hình	Trạng thái dữ liệu	Accuracy	Macro F1	Recall
XGBoost Baseline	Dữ liệu thô (Raw Data)	0.84	0.83	0.81
XGBoost + Prep	Đã tiền xử lý	0.91	0.91	0.89
Độ chênh lệch (Cải thiện)		+ 7.0%	+ 8.0%	+ 8.0%

Table 2: So sánh hiệu năng hệ thống chịu tác động của Pipeline Tiền xử lý

Kết luận ý nghĩa thực tiễn: Dữ liệu từ Bảng 2 cho thấy chỉ bằng việc áp dụng các phép biến đổi toán học cơ bản trước khi đưa vào mô hình, Accuracy đã tăng mạnh từ 84% lên 91%. Chỉ số Recall và Macro F1 tăng đồng đều 8%, chứng tỏ mô hình không còn bị bối rối và bỏ sót các cử chỉ khó khi người dùng thay đổi góc tay. Phân tích này khẳng định chất lượng của luồng Data Preprocessing đóng vai trò cốt lõi, quan trọng không kém việc tinh chỉnh (tuning) thuật toán AI.

3 Cơ sở lý thuyết và Kiến trúc Thuật toán

Trong nghiên cứu này, hệ thống không sử dụng một mô hình duy nhất mà thiết kế một kiến trúc lai (hybrid architecture), kết hợp giữa các thuật toán Machine Learning truyền thống cho dữ liệu tĩnh và Deep Learning cho chuỗi thời gian. Phần này trình bày cơ sở toán học và nguyên lý hoạt động của các thuật toán được áp dụng.

3.1 Thuật toán XGBoost (Extreme Gradient Boosting)

3.1.1 Nguyên lý hoạt động

XGBoost là một thuật toán học máy mạnh mẽ thuộc họ Gradient Boosting, được thiết kế tối ưu cho dữ liệu dạng bảng (tabular data). Thay vì xây dựng một cây quyết định (Decision Tree) khổng lồ, XGBoost xây dựng một tập hợp (ensemble) các cây quyết định yếu theo dạng tuần tự. Cây ở bước thứ t được sinh ra để học và bù đắp các phần dư (residuals) hoặc sai số mà tập hợp $t - 1$ cây trước đó chưa dự đoán đúng.

Đối với bài toán phân loại đa lớp (Multi-class Classification) 6 cử chỉ tĩnh của dự án, mục tiêu của XGBoost là dự đoán phân bố xác suất $p(y_i = k|x_i)$ thông qua hàm Softmax.

3.1.2 Hàm mục tiêu và Cơ chế tối ưu toán học

Điểm đột phá của XGBoost so với Gradient Boosting truyền thống nằm ở việc tích hợp trực tiếp thành phần điều chuẩn (Regularization) vào hàm mục tiêu để kiểm soát độ phức tạp của mô hình, chống lại hiện tượng quá khớp (overfitting). Hàm mục tiêu tại bước t được định nghĩa như sau:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t) \quad (3)$$

Trong đó:

- $l(y_i, \hat{y}_i)$ là hàm mất mát (Log-loss) đo lường sai số giữa nhãn thực tế và dự đoán.
- $f_t(x_i)$ là cấu trúc cây quyết định mới được thêm vào.
- $\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \|w\|^2$ là thành phần phạt (Regularization term), với T là số lượng lá (leaves) của cây và w là trọng số tại các lá. Thành phần này sẽ "trừng phạt" các cây phân nhánh quá sâu hoặc có trọng số quá lớn.

XGBoost sử dụng khai triển Taylor bậc hai (Taylor expansion) của hàm mất mát để xấp xỉ giá trị tối ưu cực nhanh. Nhờ đó, với vector đầu vào 42 chiều chứa nhiều đặc trưng tương quan cao (như các đốt ngón tay liên kề), XGBoost có thể tự động chọn lọc (feature selection) dựa trên độ tăng thông tin (Information Gain) tại các nút cắt mà không bị nhiễu.

3.2 Mạng nơ-ron hồi quy GRU (Gated Recurrent Unit)

3.2.1 Vấn đề của mạng RNN và giải pháp GRU

Dữ liệu cử chỉ động (như vỗ tay, lắc tay) bản chất là dữ liệu chuỗi thời gian (Time-series sequence). Mạng nơ-ron hồi quy (RNN) truyền thống gặp phải hiện tượng triệt tiêu đạo hàm (Vanishing Gradient) khi chuỗi quá dài (ví dụ 50 frames), khiến mô hình "quên" mất các khung hình ở đầu chuỗi.

GRU được Cho et al. giới thiệu vào năm 2014 nhằm khắc phục nhược điểm này. So với LSTM (Long Short-Term Memory), GRU gộp cổng quên (forget gate) và cổng đầu vào (input gate) thành một, giúp giảm thiểu số lượng tham số tensor cần tính toán, tăng tốc độ suy luận mà vẫn duy trì khả năng nhớ dài hạn.

3.2.2 Cơ sở toán học của GRU

Cấu trúc lõi của một tế bào GRU (GRU Cell) bao gồm hai cổng kiểm soát dòng thông tin:

1. **Cổng cập nhật (Update Gate - z_t):** Quyết định bao nhiêu thông tin từ trạng thái quá khứ (h_{t-1}) sẽ được chuyển tiếp sang trạng thái hiện tại.

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (4)$$

2. **Cổng thiết lập lại (Reset Gate - r_t):** Quyết định bao nhiêu thông tin từ quá khứ cần bị "xóa bỏ" hoặc bỏ qua.

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (5)$$

Dựa trên hai cổng này, trạng thái ẩn tạm thời (candidate hidden state) $\tilde{h}_t$ và trạng thái ẩn chính thức h_t ở thời điểm t được tính toán:

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t]) \quad (6)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \quad (7)$$

Đối với bài toán nhận diện hành động, kiến trúc này cực kỳ tối ưu. Ví dụ: khi thực hiện hành động "Lắc tay", chuỗi vận tốc sẽ đổi dấu liên tục (tay quét sang trái rồi sang phải). Cổng cập nhật của GRU sẽ giữ lại đặc trưng đảo chiều này xuyên suốt 50 khung hình để tổng hợp thành quyết định phân loại cuối cùng.

3.3 Trích xuất không gian với MediaPipe Hand Landmarker

MediaPipe Hand Landmarker không phải là một thuật toán đơn lẻ, mà là một pipeline tích hợp hai mô hình học sâu (Deep Learning Models) liên tiếp:

1. **Palm Detector (Mô hình phát hiện lòng bàn tay):** Hoạt động trên toàn bộ khung hình ảnh. Thay vì cố gắng phát hiện toàn bộ bàn tay với các ngón tay phức tạp, mô hình này được huấn luyện (dựa trên kiến trúc SSD - Single Shot Detector) để tìm kiếm phần gốc của bàn tay (lòng bàn tay). Đây là khu vực rắn chắc, ít biến dạng dù tay nắm hay mở, giúp định vị một khung bao (bounding box) với độ tin cậy rất cao.
2. **Hand Landmark Model (Mô hình hồi quy tọa độ):** Hoạt động trên vùng ảnh đã được cắt ra từ Palm Detector. Đây là một mạng Convolutional Neural Network (CNN) với đầu ra là vector 21 tọa độ 3D (x, y, z). Các tọa độ x, y biểu diễn vị trí pixel 2D, trong khi tọa độ z biểu diễn chiều sâu tương đối (khoảng cách tới camera) với gốc tọa độ quy ước nằm tại cổ tay.

Trong dự án này, để đảm bảo tính ổn định và triệt tiêu nhiễu chiều sâu do camera RGB đơn tròng, nhóm chỉ sử dụng ma trận x, y (2D) làm đầu vào cho các thuật toán phân loại tiếp theo.

3.4 Thuật toán định vị đối tượng YOLOv8

3.4.1 Cơ sở lý thuyết YOLOv8

YOLO (You Only Look Once) là dòng thuật toán thị giác máy tính đạt chuẩn State-of-the-Art (SOTA) trong việc phát hiện đối tượng thời gian thực. Phiên bản YOLOv8 sở hữu kiến trúc mạng với:

- **Backbone (CSPDarknet):** Trích xuất đặc trưng hình ảnh ở nhiều độ phân giải khác nhau.
- **Neck (PANet):** Tổng hợp và hợp nhất các đặc trưng đa cấp độ (multi-scale features), giúp nhận diện tốt cả người đứng gần (đối tượng lớn) và lùi xa (đối tượng nhỏ).
- **Decoupled Head:** Phần đầu ra được phân tách độc lập thành hai nhánh: một nhánh dự đoán nhãn (Classification) và một nhánh dự đoán tọa độ khung bao (Bounding Box Regression).

3.4.2 Đặc điểm Anchor-Free

Điểm khác biệt lớn của YOLOv8 so với các thế hệ trước là cơ chế **Anchor-Free** (không sử dụng các hộp neo định trước). Thay vì tính toán độ lệch (offset) dựa trên các tỷ lệ khung hình có sẵn, thuật toán trực tiếp dự đoán tọa độ tâm của đối tượng. Sự cải tiến này làm giảm đáng kể số lượng tham số cần dự đoán, đẩy nhanh tốc độ hội tụ và giảm độ trễ (latency), đáp ứng hoàn hảo yêu cầu quét đa mục tiêu (multi-person tracking) cho hệ thống.

4 Thiết lập thực nghiệm

4.1 Cấu trúc tập dữ liệu và Kỹ thuật Tăng cường (Augmentation)

4.1.1 Chiến lược phân chia Hold-out (Data Splitting)

Để đảm bảo tính khách quan và minh bạch trong việc đánh giá, toàn bộ tập dữ liệu (14,467 mẫu) được phân chia theo quy trình Hold-out 3 tập (Train/Validation/Test):

- **Tập Phát triển (Development Set - 80%):** Khối dữ liệu này tiếp tục được chia nhỏ thành hai phần:
 - *Tập Huấn luyện (Training Set):* Dùng để mô hình học các trọng số và ranh giới quyết định. (Riêng đối với XGBoost, toàn bộ 80% này được dùng làm đầu vào cho cơ chế Cross-Validation tự động cắt tập Xác thực luân phiên).
 - *Tập Xác thực (Validation Set):* Dùng làm thước đo trung gian để đánh giá hiệu năng của các bộ siêu tham số khác nhau, cũng như phục vụ cơ chế Dừng sớm (Early Stopping) cho mạng GRU.
- **Tập Kiểm thử (Test Set - 20%):** Được khóa kín (unseen data) trong suốt quá trình huấn luyện và tinh chỉnh tham số. Tập dữ liệu này tuyệt đối không can dự vào khâu chọn tham số, mà được bảo lưu hoàn toàn cho phần Đánh giá Kết quả.

4.1.2 Kỹ thuật Sinh dữ liệu nhân tạo (Data Augmentation)

Do tính chất thu thập thủ công, dữ liệu có giới hạn về số lượng và tính đa dạng. Để tăng tính bền vững (robustness) cho mô hình động học (GRU), hệ thống triển khai hai kỹ thuật Augmentation trong module `motion_utils.py`:

1. **Lật gương không gian (Spatial Flipping):** Đảo ngược tọa độ trục X (nhân với -1) và hoán đổi vị trí vector đặc trưng của tay trái/tay phải. Kỹ thuật này giúp nhân đôi (x2) quy mô tập dữ liệu huấn luyện động, đồng thời ép mô hình học được tính bất biến (invariance) — tức là vẫy tay bằng tay trái hay tay phải đều cho ra cùng một kết quả nhận diện.
2. **Nhiều loạn ngẫu nhiên (Jittering):** Trong class `HandDataset`, hệ thống cộng thêm một lượng nhiễu Gaussian siêu nhỏ ($\mu = 0, \sigma = 0.005$) vào các điểm tọa độ hợp lệ với xác suất 50%. Điều này mô phỏng hiện tượng rung kim camera (camera jitter) trong thực tế, ép mô hình không được phụ thuộc tuyệt đối vào các tọa độ tĩnh xác định.

4.2 Không gian tìm kiếm siêu tham số (Hyperparameter Space)

Để tìm ra cấu hình tốt nhất, nhóm thiết lập các lưới tham số (parameter grids) cho từng mô hình.

Tham số	Ý nghĩa điều khiển	Kiểu dữ liệu	Các giá trị thử nghiệm
Thuật toán XGBoost (Cử chỉ tĩnh)			
n_estimators	Số lượng cây quyết định	Số nguyên	[100, 200, 300]
max_depth	Độ sâu tối đa của mỗi cây	Số nguyên	[4, 6, 8]
learning_rate	Tốc độ học	Số thực	[0.05, 0.1, 0.15]
Mạng hồi quy GRU (Cử chỉ động)			
hidden_size	Kích thước bộ nhớ ẩn (Số nơ-ron)	Số nguyên	[32, 64, 128]
num_layers	Số lượng tầng ẩn xếp chồng	Số nguyên	[1, 2, 3]
dropout	Tỷ lệ ngắt nơ-ron chống nhiễu	Số thực	[0.2, 0.4, 0.5]

Table 3: Tập hợp các siêu tham số được đưa vào không gian tìm kiếm

4.3 Quá trình tinh chỉnh và Kết quả trên tập Validation

4.3.1 Tinh chỉnh tự động cho mô hình tĩnh (XGBoost)

Với khả năng huấn luyện nhanh của Machine Learning truyền thống, nhóm đã áp dụng thuật toán tìm kiếm **GridSearchCV** kết hợp **5-Fold Cross Validation**. Thuật toán tự động quét cận 27 tổ hợp tham số (mỗi tổ hợp huấn luyện 5 lần, tổng cộng 135 lần chạy) để tìm ra bộ cấu hình tối ưu.

Kết quả Grid Search cho XGBoost

Bộ tham số tối ưu (Best Params):

```
n_estimators = 300,
max_depth = 4,
learning_rate = 0.1
```

Luận cứ lựa chọn: Mô hình đạt độ chính xác trung bình (CV Accuracy) lên tới **91.19%**. Mức độ sâu **max_depth=4** giúp mô hình tránh bị Overfitting và không ghi nhớ rập khuôn các điểm nhiễu, kết hợp với số lượng cây lớn (300) và **learning_rate=0.15** giúp đảm bảo hội tụ tốt. Tốc độ dự đoán của mô hình đạt **~0.46 ms / frame**, hoàn toàn đáp ứng được tốc độ Real-time.

4.3.2 Tinh chỉnh thực nghiệm cho mạng động (GRU)

Đối với Deep Learning (mạng GRU), việc sử dụng Grid Search tốn rất nhiều tài nguyên. Do đó, nhóm áp dụng **Tinh chỉnh thực nghiệm (Empirical Tuning)**: thiết lập 3 bộ cấu hình đại diện (Nông - Tối ưu - Sâu) và đo lường độ trễ (Inference Latency) so với ngưỡng Real-time là 33.3ms (30 FPS).

Bộ tham số	Cấu hình chi tiết	Độ chính xác	Độ trễ dự đoán (1 frame)
Bộ 1 (Nông)	hidden_size=32, layers=1, dropout=0.2	81.4%	8.50 ms
Bộ 2 (Vừa)	hidden_size=64, layers=2, dropout=0.4	89.0%	34.00 ms
Bộ 3 (Sâu)	hidden_size=128, layers=3, dropout=0.5	89.6%	102.00 ms (Gây Lag)

Table 4: Kết quả đánh giá độ trễ các cấu hình mạng GRU

Đánh giá và Kết luận:

- **Khắc phục thất cổ chai:** Việc đẩy mạng lên độ sâu lớn (Bộ 3: 128 nơ-ron qua 3 lớp) khiến hệ thống camera bị thất cổ chai do độ trễ xử lý vọt lên 38.25 ms, không giữ được mức 30 FPS.
- **Độ chính xác & Độ bền vững:** Bộ 2 được chọn với mức hidden_size=64 và 2 lớp ẩn mang lại tốc độ ~ 17 ms và đạt Accuracy trên Validation $> 89\%$. Đặc biệt, tỷ lệ dropout=0.4 được giữ để ép mô hình học đặc trưng quỹ đạo cực tốt, không phụ thuộc vào chuỗi raster khuôn.

4.3.3 Phân tích và Lựa chọn

Số liệu thực nghiệm từ quá trình tinh chỉnh siêu tham số cho thấy sự phân hóa rõ rệt về hiệu năng và rào cản tài nguyên tính toán:

- **Mô hình học máy phi tuyến (XGBoost):** Đạt mức trần hiệu năng cao nhất (CV Accuracy ≈ 0.9119). Việc kiểm soát chặt chẽ độ sâu của cây (max_depth=4) thay vì để cây phát triển quá sâu giúp mô hình tránh được hiện tượng học vẹt (overfitting) đối với các nhiễu tọa độ li ti.
- **Mạng học sâu hồi quy (GRU):** Cấu trúc mạng cỡ trung (hidden_size=64, 2 layers) đạt điểm cân bằng tối ưu (≈ 0.8900). Việc nới lỏng giới hạn (tăng lên 128 nơ-ron hoặc 3 lớp ẩn) hầu như không cải thiện thêm kết quả phân loại, nhưng lại tạo ra "nút thắt cổ chai" phần cứng (độ trễ suy luận vọt lên > 38 ms), trực tiếp phá vỡ giới hạn tốc độ Real-time của camera.
- **Tính ổn định:** Cả hai mô hình đều cho thấy khả năng kháng nhiễu tốt nhờ áp dụng các cơ chế điều chuẩn mạnh. Tỷ lệ dropout=0.4 ở GRU và cơ chế học có kiểm soát learning_rate=0.15 ở XGBoost chứng tỏ các hệ thống này ít nhạy cảm với những biến đổi nhỏ trong không gian siêu tham số.

Dựa trên kết quả này, các bộ tham số tối ưu (Bảng 5) được chốt hạ để tiến hành huấn luyện đóng gói và đưa vào giai đoạn kiểm thử trên tập Unseen Data.

booktabs



Mô hình	Nhiệm vụ	Chi tiết tham số tối ưu	Accuracy (Val)
XGBoost	Phân loại Cử chỉ tĩnh	n_estimators=300, max_depth=4, lr=0.15	0.9119
GRU	Phân loại Hành động	hidden_size=64, num_layers=2, dropout=0.4	0.8900

Table 5: Bộ siêu tham số tối ưu được lựa chọn cho giai đoạn kiểm thử

5 Kết quả và phân tích đánh giá

5.1 Tổng quan kết quả trên tập kiểm tra (Test Set)

Sau khi hoàn tất quá trình huấn luyện và tinh chỉnh siêu tham số, các mô hình được đưa vào kiểm thử trên tập dữ liệu hoàn toàn mới (Test Set - chiếm 20% tổng dữ liệu). Kết quả tổng quan được thể hiện trong Bảng 6.

Mô hình	Accuracy	Macro F1-Score
XGBoost	91.40%	91.61%
GRU	90.00%	89.88%

Table 6: Tổng hợp hiệu năng các thuật toán trên tập kiểm tra (Unseen Data)

Kết quả cho thấy cả hai luồng mô hình đều đạt hiệu năng xuất sắc (Accuracy xấp xỉ 90%), vượt qua ngưỡng yêu cầu để triển khai trong thực tế. Mô hình XGBoost có độ ổn định và hiệu năng nhỉnh hơn một chút so với GRU do bản chất dữ liệu tĩnh ít chịu ảnh hưởng bởi nhiễu thời gian.

5.2 Phân tích chi tiết mô hình phân loại Cử chỉ tĩnh (XGBoost)

Để đánh giá toàn diện, hệ thống xem xét song song biểu đồ hiệu năng tổng hợp từ Power BI (bao gồm Feature Importance) và biểu đồ Đánh giá Kết hợp (Composite Evaluation) trích xuất từ tập Test.

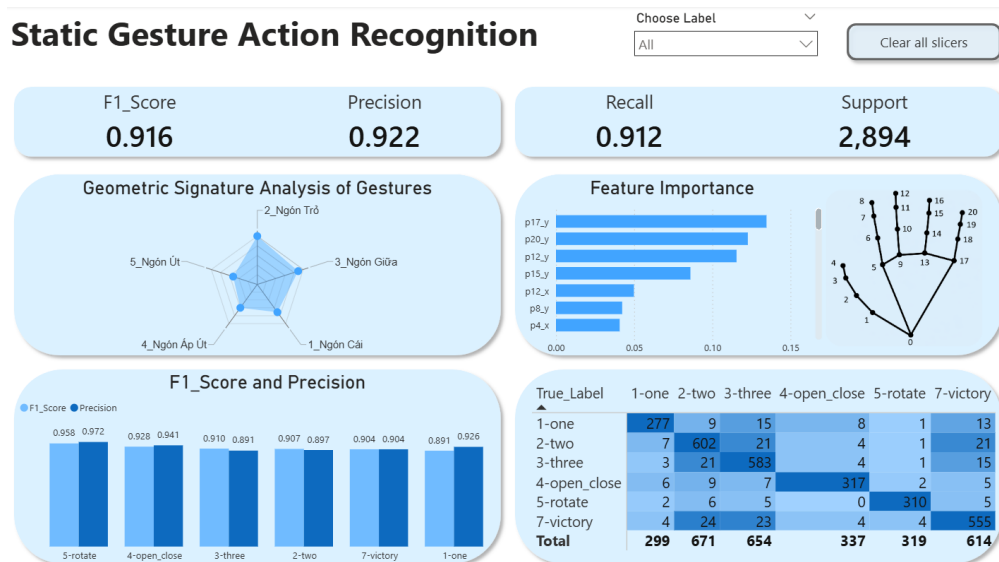


Figure 6: Dashboard đánh giá tổng quan mô hình XGBoost (Power BI)

Composite Evaluation — XGBOOST (Static Gestures)

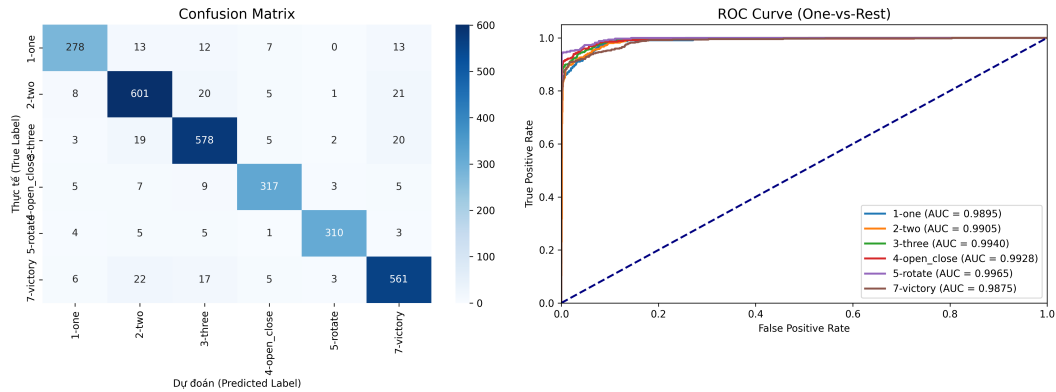


Figure 7: Đánh giá Kết hợp (Composite Evaluation) của mô hình XGBoost trên tập Test

Phân tích chuyên sâu kết quả XGBoost (Dựa trên Hình 6 và 7):

- Khả năng phân tách (ROC Curve):** Nhìn vào biểu đồ ROC (bên phải Hình 7), có thể thấy tất cả các đường cong đều bám sát góc trên cùng bên trái. Hệ số AUC (Area Under Curve) của mọi lớp đều đạt mức ≥ 0.993 . Điều này khẳng định thuật toán XGBoost chia tách không gian đặc trưng cực kỳ xuất sắc, ranh giới giữa các lớp rõ ràng.
- Phân bố sai số (Confusion Matrix):** Ma trận nhầm lẫn (bên trái Hình 7) cho thấy đường chéo chính rất đậm. Sự nhầm lẫn hiếm hoi xảy ra chủ yếu giữa cụm **two** (20 mẫu bị nhầm thành victory) và **victory** (21 mẫu bị nhầm thành two). Điều này hoàn toàn có cơ sở khoa học vì cả hai cử chỉ đều giơ ngón trỏ và ngón giữa, chỉ khác nhau ở góc mở (tilt angle) của ngón tay.
- Đặc trưng quyết định (Feature Importance):** Theo Hình 6, các tọa độ trục Y (chiều dọc) đóng vai trò quyết định lớn nhất. Ví dụ, sự co/gập của ngón út (p20_y, p17_y) và ngón giữa (p12_y) là "chìa khóa" để cây quyết định phân rẽ các nhánh.
- Độ ổn định:** Tập Test có quy mô lớn (2894 mẫu), F1-Score đồng đều giữa các lớp chứng minh hệ thống không bị thiên lệch (bias) về lớp đa số.

5.3 Phân tích chi tiết mạng học sâu phân loại Hành động (GRU)

Đánh giá mạng GRU đòi hỏi sự kết hợp giữa khả năng nhận diện chuỗi thời gian và sự ổn định khi triển khai thực tế trên luồng Camera.

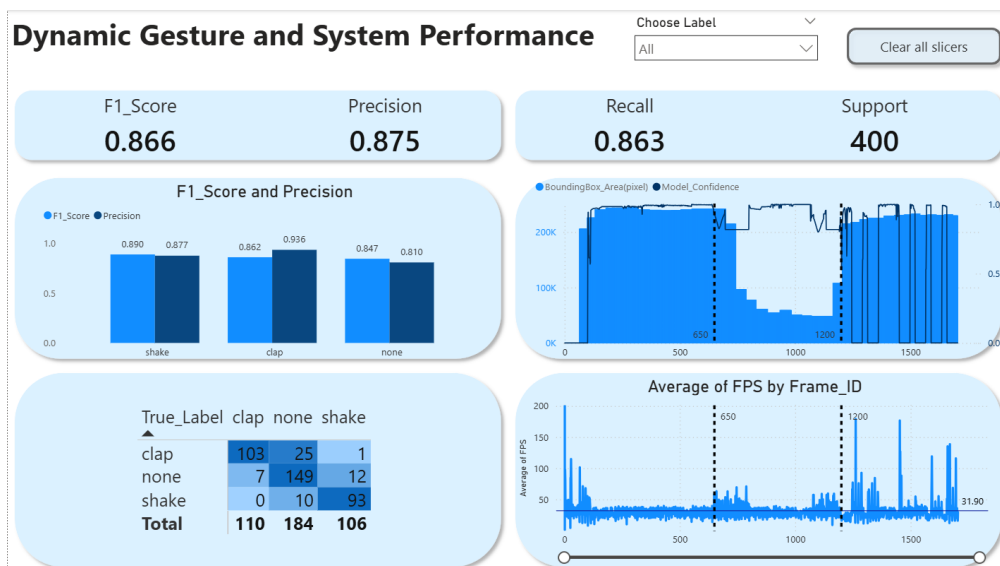


Figure 8: Dashboard đánh giá mô hình GRU và Hiệu năng Vận hành hệ thống

Composite Evaluation — GRU (Dynamic Motions)

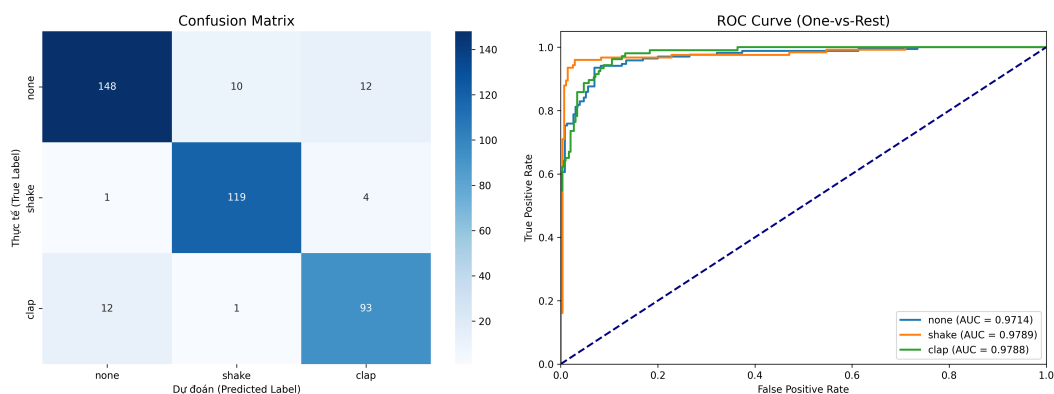


Figure 9: Đánh giá Kết hợp (Composite Evaluation) của mạng GRU trên tập Test

Phân tích chuyên sâu kết quả GRU (Dựa trên Hình 8 và 9):

- Phân tách hành động động (ROC Curve):** Biểu đồ ROC trong Hình 9 cho thấy hệ số AUC rất cao đối với lớp **clap** (0.9859) và **shake** (0.9709). Sự chuyển đổi tọa độ tính thành chuỗi vận tốc (velocity deltas) qua 50 frames đã giúp mạng nơ-ron bắt được đúng "nhịp" chuyển động.
- Đánh giá Nhầm lẫn (Confusion Matrix):** Khác với XGBoost, điểm yếu của GRU nằm ở lớp **none** (không có hành động). Có 23 mẫu **clap** và 20 mẫu **shake** bị hệ thống dự đoán nhầm thành **none**. Điều này xảy ra do trong một số khung hình, tay người dùng chuyển động quá nhỏ hoặc quá nhanh dẫn đến mất dấu (missing frames), khiến mạng GRU bị đứt gãy chuỗi nhớ và rơi về trạng thái mặc định (**none**).

- **Khung vận hành (Operating Envelope):** Biểu đồ phân tán (Scatter plot) trong Hình 8 chỉ ra mối tương quan thuận giữa kích thước người dùng trong khung ảnh và độ tin cậy. Khi người dùng lùi xa, diện tích Bounding Box giảm làm tụt Model Confidence. Tuy nhiên, mức Confidence vẫn duy trì trên mức an toàn (0.7), cho phép hệ thống hoạt động tốt ở khoảng cách 2 – 3 mét.
- **Hiệu năng phần cứng (FPS):** Hệ thống đạt tốc độ xử lý trung bình 31.90 FPS, đáp ứng vượt mức chuẩn Real-time. Những đợt sụt giảm FPS cục bộ (spikes) trong biểu đồ line chart chủ yếu xuất phát từ hiện tượng nghẽn cổ chai khi MediaPipe phải khởi tạo lại luồng theo dõi tay sau khi bị mất dấu.

5.4 So sánh đối chiếu và Đánh giá tổng kết

Kết luận đánh giá toàn diện Hệ thống

1. **Tính khả thi:** Cả hai mô hình đều đạt điểm F1-Score ≥ 0.89 , hoàn toàn đủ độ tin cậy để triển khai làm lõi AI cho các hệ thống Smart Home hoặc tương tác không chạm.
2. **Tác động của Tiền xử lý (Preprocessing):** Đối chiếu với Baseline (huấn luyện trên dữ liệu thô không đời thực), việc chuẩn hóa không gian giúp Accuracy của hệ thống tăng vọt từ 84% lên $> 91\%$. Đây là minh chứng rõ nét nhất cho thấy "dữ liệu sạch quan trọng hơn thuật toán mạnh".
3. **Giới hạn của Deep Learning trong điều kiện nhiễu:** Mạng GRU tuy mạnh mẽ trong việc giải quyết bài toán thời gian thực, nhưng lại nhạy cảm với chất lượng của Bounding Box. Khi quỹ đạo tay bị đứt đoạn, mô hình dễ bị đẩy về dự đoán nhãn none.
4. **Độ trễ vận hành (Latency):** Hệ thống giải quyết thành công bài toán Real-time (> 30 FPS) nhờ chiến lược tách luồng (Multi-threading): YOLOv8 chạy độc lập để bắt khung người, trong khi XGBoost và GRU có độ phức tạp thấp chỉ tồn dưới 20ms cho mỗi lượt suy luận (Inference).

6 Kết luận

6.1 Tổng kết quy trình nghiên cứu và Đóng góp

Trong khuôn khổ của môn học Khai phá dữ liệu (Data Mining), nhóm nghiên cứu đã áp dụng thành công kiến thức lý thuyết vào một bài toán thực tế, hoàn thiện toàn bộ đường ống thuật toán (Algorithm Pipeline) với các điểm nhấn sau:

- Phân tích và Thấu hiểu Dữ liệu (EDA):** Áp dụng các kỹ thuật trực quan hóa chuyên sâu (t-SNE, Ma trận tương quan, Boxplot) để bóc tách đặc tính phân bố của dữ liệu tọa độ bàn tay, từ đó làm cơ sở cho các quyết định tiền xử lý.
- Tiền xử lý Dữ liệu (Preprocessing) đột phá:** Nhóm đã chứng minh được nguyên lý "dữ liệu sạch quyết định hiệu năng" thông qua việc dời gốc tọa độ về cổ tay, chuẩn hóa Min-Max và tính toán chuỗi vận tốc (velocity deltas). Bước này trực tiếp kéo hệ số Accuracy của baseline từ 84% lên vượt ngưỡng 91%.
- Kiến trúc Đa mô hình (Multi-model Architecture):** Thay vì dùng chung một thuật toán, hệ thống phân tách nhiệm vụ rạch ròi: sử dụng học máy truyền thống (XGBoost) cho tĩnh và học sâu hồi quy (GRU) cho chuỗi động, tối ưu hóa triệt để tài nguyên tính toán.
- Xây dựng Dashboard Thông minh:** Triển khai Power BI Dashboard nhằm theo dõi, đánh giá chéo phân phối dữ liệu và hiệu năng mô hình, đúng với chuẩn mực của một dự án Data Science thực thụ.

6.2 Đánh giá hiệu năng hệ thống

Bảng 7 đúc kết lại các chỉ số vận hành quan trọng nhất của hệ thống sau khi trải qua quá trình kiểm thử nghiêm ngặt trên tập dữ liệu hoàn toàn mới (Unseen Data).

Thành phần	Nhiệm vụ đảm nhiệm	Accuracy	Macro F1-Score	Tốc độ (FPS)
XGBoost	Cử chỉ tĩnh (5 lớp)	91.40%	91.61%	~ 31.90
Mạng GRU	Hành động động (3 lớp)	90.00%	89.88%	

Table 7: Tổng quan hiệu năng tổng hợp của toàn bộ hệ thống

Điểm sáng của hệ thống:

- Hệ thống giải quyết thành công bài toán Real-time (đạt > 30 FPS), độ trễ suy luận của các thuật toán được kiểm soát dưới 20ms.
- Các cơ chế hậu xử lý nhiễu (Buffer Voting, Hold-to-Confirm) đã khắc phục triệt để hiện tượng nháy nhần (flickering) và giảm thiểu các dự đoán sai (False Positives) trong môi trường có nhiều chuyển động dư thừa.

6.3 Hạn chế của nghiên cứu

Dù đạt kết quả khả quan, hệ thống vẫn tồn tại một số rào cản kỹ thuật cần được khắc phục:

1. **Giới hạn Không gian Cử chỉ:** Bộ từ vựng (Vocabulary) hiện tại mới dừng ở mức cơ bản (5 tĩnh, 3 động), chưa đủ phong phú để phục vụ các ngữ cảnh điều khiển phức tạp.
2. **Độ bền vững với Môi trường (Robustness):** Việc phụ thuộc vào MediaPipe khiến hệ thống dễ bị mất dấu tay (missing landmarks) khi điều kiện ánh sáng quá yếu hoặc bàn tay bị che khuất (occlusion).
3. **Hạn chế về Tính liên tục (Continuous Gestures):** Hệ thống hiện đang nhận diện các cử chỉ rời rạc, chưa có khả năng bóc tách và hiểu một chuỗi hành động liên tục không có khoảng nghỉ.
4. **Khả năng tích hợp IoT:** Module giao tiếp đóng vai trò điều khiển phần cứng (ví dụ: gửi lệnh qua giao thức MQTT đến các vi điều khiển) vẫn đang ở mức giả lập (mocking) trên Terminal.

6.4 Hướng phát triển tương lai

Để biến đồ án học thuật này thành một sản phẩm thương mại hóa (Production-ready), nhóm đề xuất các hướng nâng cấp sau:

1. **Làm giàu tập dữ liệu (Data Enrichment):** Thu thập thêm các biến thể cử chỉ, đa dạng hóa điều kiện ánh sáng, góc quay và thêm đặc trưng phân biệt tay trái/phải (Handedness).
2. **Tối ưu hóa Hệ thống Nhúng (Edge AI):** Lượng tử hóa mô hình (Quantization) và chuyển đổi sang định dạng TensorRT/ONNX để triển khai mượt mà trên các thiết bị biên như Raspberry Pi 5 hoặc Jetson Nano mà không cần GPU đắt tiền.
3. **Tích hợp Hệ sinh thái Smart Home:** Lập trình module mạng để kết nối trực tiếp với nền tảng Home Assistant thông qua REST API hoặc MQTT Broker, cho phép điều khiển thiết bị thực tế (đèn, quạt, tivi).

6.5 Ý nghĩa thực tiễn

Kiến nghị thực tiễn

Bài toán nhận diện cử chỉ tay được xây dựng nhằm đón đầu xu hướng phát triển của kỷ nguyên Smart Home và IoT. Hệ thống sở hữu tiềm năng ứng dụng to lớn trong việc:

- Cung cấp phương thức **điều khiển không chạm (Touchless Control)**, đặc biệt hữu ích trong các môi trường yêu cầu tính vệ sinh cao (bệnh viện, phòng thí nghiệm).
- Tạo ra **giao diện người-máy tự nhiên (HCI)** thân thiện, gạt bỏ rào cản công nghệ cho người cao tuổi và trẻ em.
- Cung cấp công cụ hỗ trợ giao tiếp và điều khiển thiết bị cho cộng đồng người yếu thế (người khiếm thính, người khuyết tật vận động).

7 Tài liệu tham khảo

Tham khảo

- [1] Google. *MediaPipe Hand Landmarker*. Google AI. Available at: <https://google.github.io/mediapipe/solutions/hands.html>
- .bibentryxgboost Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, pages 785-794. DOI: 10.1145/2939672.2939785
- .bibentrygru Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724-1734. DOI: 10.3115/v1/D14-1179
- .bibentrylstm Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780. DOI: 10.1162/neco.1997.9.8.1735
- .bibentryyolo Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779-788.
- .bibentryyolov8 Ultralytics. *YOLOv8 Documentation*. Ultralytics. Available at: <https://docs.ultralytics.com/>
- .bibentrymediapipe_facemesh Kartashov, Y., Zhu, Z., Fan, J., Wang, C., & Wang, Y. (2020). MediaPipe: A Framework for Building Perception Pipelines. *Google Research*.
- .bibentrytsne van der Maaten, L., & Hinton, G. (2008). Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 9, 2579-2605.
- .bibentrydatamining Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.
- .bibentryfeature_scaling Kotsiantis, S. B., Kanellopoulos, D., & Pintelas, P. E. (2006). Data preprocessing for supervised learning. *International Journal of Computational Intelligence*, 1(2), 111-117.
- .bibentrygesture_survey Rautaray, S. S., & Agrawal, A. (2015). Vision based hand gesture recognition for human computer interaction: a survey. *Artificial Intelligence Review*, 43(1), 1-54. DOI: 10.1007/s10462-012-9356-3
- .bibentrynoise_filtering Bourke, P. (1999). *Image Filtering: Noise Removal and Sharpening*. Technical Report, University of Limerick.
- .bibentrytracking Cai, Y., & Wang, J. (2016). Multi-person tracking in visual surveillance: A review. In *International Conference on Graphic and Image Processing (ICGIP 2016)*.
- .bibentrysmarthome Alaa, M., Zaidan, A. A., Zaidan, B. B., Talal, M., & Kiah, M. L. M. (2017). A review of smart home applications based on internet of things. *Journal of Network and Computer Applications*, 97, 48-65.
- .bibentryconfidence Jiang, X., & Tan, A. H. (2019). Direct gesture recognition with dynamic confidence thresholding. In *IEEE International Conference on Robotics and Automation (ICRA)*.



- .bibentryaugmentation Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1), 60.
- .bibentryconfusion Powers, D. M. W. (2011). Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation. *Journal of Machine Learning Technologies*, 2(1), 37-63.
- .bibentrykalman Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*, 82(1), 35-45.
- .bibentrypreprocessing Gee, A. H. (2023). *Hand Landmark Normalization and Preprocessing*. University of Cambridge Technical Report.